Experiment 8

Aim

To implement the SHA-512 cryptographic hash function in Java to generate a 512-bit hash value for a given input message.

Theory

SHA-512 (Secure Hash Algorithm 512) is a cryptographic hash function belonging to the SHA-2 family. It accepts an input message of arbitrary length and produces a fixed-size hash value of 512 bits (64 bytes), typically represented as 128 hexadecimal characters.

SHA-512 provides the following properties:

- **Fixed-length output:** Produces a 512-bit hash regardless of the input length.
- **Deterministic:** The same input always produces the same hash.
- **One-way property:** It is computationally infeasible to recover the original message from its hash.
- **Avalanche effect:** A small change in the input can significantly change the resulting hash.

SHA-512 processes the message in 1024-bit blocks using padding, message scheduling, and multiple rounds of bitwise operations.

In Java, the MessageDigest class from the java.security package provides a built-in implementation of SHA-512.

Program

```
import java.util.Scanner;

public class SHA512 {
    private static final long[] K = {
        0x428a2f98d728ae22L, 0x7137449123ef65cdL,
        0xb5c0fbcfec4d3b2fL, 0xe9b5dba58189dbbcL,
        0x3956c25bf348b538L, 0x59f111f1b605d019L,
        0x923f82a4af194f9bL, 0xab1c5ed5da6d8118L,
        0xd807aa98a3030242L, 0x12835b0145706fbeL,
        0x243185be4ee4b28cL, 0x550c7dc3d5ffb4e2L,
        0x72be5d74f27b896fL, 0x80deb1fe3b1696b1L,
        0x9bdc06a725c71235L, 0xc19bf174cf692694L,
        0xe49b69c19ef14ad2L, 0xefbe4786384f25e3L,
        0x0fc19dc68b8cd5b5L, 0x240ca1cc77ac9c65L,
        0x2de92c6f592b0275L, 0x4a7484aa6ea6e483L,
        0x5cb0a9dcbd41fbd4L, 0x76f988da831153b5L,
        0x983e5152ee66dfabL, 0xa831c66d2db43210L,
        0xb00327c898fb213fL, 0xbf597fc7beef0ee4L,
        0xc6e00bf33da88fc2L, 0xd5a79147930aa725L,
        0x06ca6351e003826fL, 0x142929670a0e6e70L,
        0x27b70a8546d22ffcL, 0x2e1b21385c26c926L,
        0x4d2c6dfc5ac42aedL, 0x53380d139d95b3dfL,
        0x650a73548baf63deL, 0x766a0abb3c77b2a8L,
        0x81c2c92e47edaee6L, 0x92722c851482353bL,
        0xa2bfe8a14cf10364L, 0xa81a664bbc423001L,
        0xc24b8b70d0f89791L, 0xc76c51a30654be30L,
        0xd192e819d6ef5218L, 0xd69906245565a910L,
        0xf40e35855771202aL, 0x106aa07032bbd1b8L,
        0x19a4c116b8d2d0c8L, 0x1e376c085141ab53L,
        0x2748774cdf8eeb99L, 0x34b0bcb5e19b48a8L,
```

```
            0x391c0cb3c5c95a63L, 0x4ed8aa4ae3418acbL,
            0x5b9cca4f7763e373L, 0x682e6ff3d6b2b8a3L,
            0x748f82ee5defb2fcL, 0x78a5636f43172f60L,
            0x84c87814a1f0ab72L, 0x8cc702081a6439ecL,
            0x90befffa23631e28L, 0xa4506cebde82bde9L,
            0xbef9a3f7b2c67915L, 0xc67178f2e372532bL,
            0xca273eceea26619cL, 0xd186b8c721c0c207L,
            0xeada7dd6cde0eb1eL, 0xf57d4f7fee6ed178L,
            0x06f067aa72176fbaL, 0x0a637dc5a2c898a6L,
            0x113f9804bef90daeL, 0x1b710b35131c471bL,
            0x28db77f523047d84L, 0x32caab7b40c72493L,
            0x3c9ebe0a15c9bebcL, 0x431d67c49c100d4cL,
            0x4cc5d4becb3e42b6L, 0x597f299cfc657e2aL,
            0x5fcb6fab3ad6faecL, 0x6c44198c4a475817L
        };

        private static final long[] H = {
            0x6a09e667f3bcc908L, 0xbb67ae8584caa73bL,
            0x3c6ef372fe94f82bL, 0xa54ff53a5f1d36f1L,
            0x510e527fade682d1L, 0x9b05688c2b3e6c1fL,
            0x1f83d9abfb41bd6bL, 0x5be0cd19137e2179L
        };

        private static long rotr(long x, int n) {
            return (x >>> n) | (x << (64 - n));
        }

        private static long ch(long x, long y, long z) {
            return (x & y) ^ (~x & z);
        }

        private static long maj(long x, long y, long z) {
            return (x & y) ^ (x & z) ^ (y & z);
        }

        private static long bigSigma0(long x) {
            return rotr(x, 28) ^ rotr(x, 34) ^ rotr(x, 39);
        }

        private static long bigSigma1(long x) {
            return rotr(x, 14) ^ rotr(x, 18) ^ rotr(x, 41);
        }

        private static long smallSigma0(long x) {
            return rotr(x, 1) ^ rotr(x, 8) ^ (x >>> 7);
        }

        private static long smallSigma1(long x) {
            return rotr(x, 19) ^ rotr(x, 61) ^ (x >>> 6);
        }

        public static String sha512(String message) {
            byte[] input =
    message.getBytes(java.nio.charset.StandardCharsets.UTF_8);

            long bitLength = (long) input.length * 8;
            int paddingLength = (112 - (input.length + 1) % 128 + 128) % 128;
            byte[] padded = new byte[input.length + 1 + paddingLength + 16];

            System.arraycopy(input, 0, padded, 0, input.length);
            padded[input.length] = (byte) 0x80;

            for (int i = 0; i < 8; i++) {
```

```
            padded[padded.length - 8 + i] =
                (byte) (bitLength >>> (56 - 8 * i));
        }

        long[] hash = H.clone();
        long[] w = new long[80];

        for (int offset = 0; offset < padded.length; offset += 128) {
            for (int i = 0; i < 16; i++) {
                long value = 0;
                for (int j = 0; j < 8; j++) {
                    value = (value << 8)
                          | (padded[offset + i * 8 + j] & 0xffL);
                }
                w[i] = value;
            }

            for (int i = 16; i < 80; i++) {
                w[i] = smallSigma1(w[i - 2]) + w[i - 7]
                     + smallSigma0(w[i - 15]) + w[i - 16];
            }

            long a = hash[0], b = hash[1];
            long c = hash[2], d = hash[3];
            long e = hash[4], f = hash[5];
            long g = hash[6], h = hash[7];

            for (int i = 0; i < 80; i++) {
                long t1 = h + bigSigma1(e) + ch(e, f, g)
                        + K[i] + w[i];
                long t2 = bigSigma0(a) + maj(a, b, c);

                h = g;
                g = f;
                f = e;
                e = d + t1;
                d = c;
                c = b;
                b = a;
                a = t1 + t2;
            }

            hash[0] += a;
            hash[1] += b;
            hash[2] += c;
            hash[3] += d;
            hash[4] += e;
            hash[5] += f;
            hash[6] += g;
            hash[7] += h;
        }

        StringBuilder result = new StringBuilder();

        for (long value : hash) {
            for (int i = 7; i >= 0; i--) {
                int v = (int) (value >>> (i * 8)) & 0xff;
                result.append("0123456789abcdef".charAt(v >>> 4));
                result.append("0123456789abcdef".charAt(v & 0x0f));
            }
        }

        return result.toString();
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the message: ");
        String message = sc.nextLine();

        System.out.println("SHA-512 Hash: " + sha512(message));

        sc.close();
    }
}
```

Output

```
$ java SHA512.java
Enter the message: Hello
SHA-512 Hash: 3615f80c9d293ed7402687f94b22d58e529b8cc7916f8fac7fddf7
fbd5af4cf777d3d795a7a00a16bf7e7f3fb9561ee9baae480da9fe7a18769e71886b
03f315
```

Conclusion

The SHA-512 hash function was successfully implemented in Java using the MessageDigest class. The program generates a fixed-length 512-bit hash for the given message. This experiment demonstrates the use of cryptographic hashing for data integrity and secure message verification.